

El problema: picos masivos de inscripciones

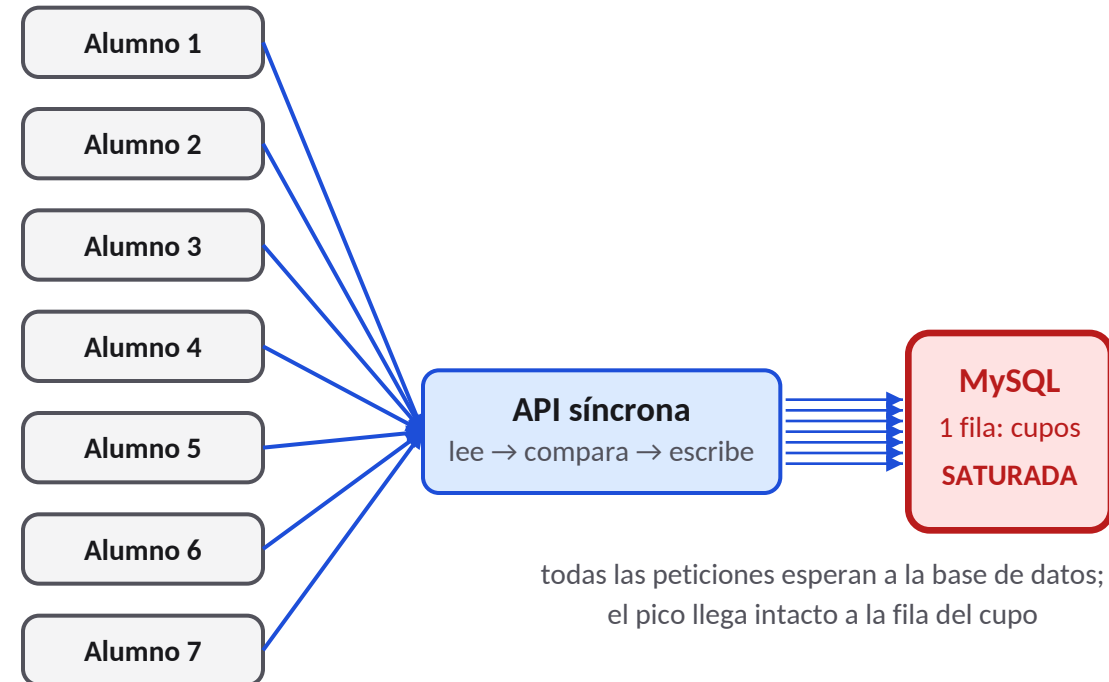
A la hora de apertura, **miles de alumnos** intentan inscribirse en los mismos cursos en el mismo minuto (*flash crowd*). Es el mismo colapso que sufre **SIU Guaraní** cada inicio de cuatrimestre.

En una arquitectura **síncrona**, cada POST lee y escribe el cupo directamente en la base de datos:

- Miles de transacciones sobre **la misma fila** (el cupo del curso).
- Se agotan las conexiones, crece la latencia hasta el *timeout*.
- El usuario reintenta y **multiplica** la carga.
- Entre leer y escribir se cuelan otras peticiones: **overbooking** y dobles inscripciones.

Objetivo: absorber el pico sin saturar la base de datos y garantizar que **nunca** se asignen más cupos que los disponibles ni se inscriba dos veces al mismo alumno.

Arquitectura síncrona bajo un pico



La solución: Queue-Based Load Leveling

1 La API registra la intención

INSERT en estado **pending** (fila nueva, sin contención) y responde **202 Accepted** con la URL de estado. Milisegundos, escala horizontalmente.

2 La cola amortigua el pico

RabbitMQ encola de forma **durable** y en orden. La base de datos ve un flujo constante, nunca el pico.

3 El worker decide de a una

UPDATE cursos SET cupos = cupos - 1 WHERE cupos > 0 en una transacción InnoDB: 1 fila afectada → **confirmed**, 0 → **rejected**. Atómico: sin overbooking.

4 El alumno consulta el resultado

GET /inscripciones/:id con Retry-After → pending · confirmed · rejected · failed.

Idempotencia

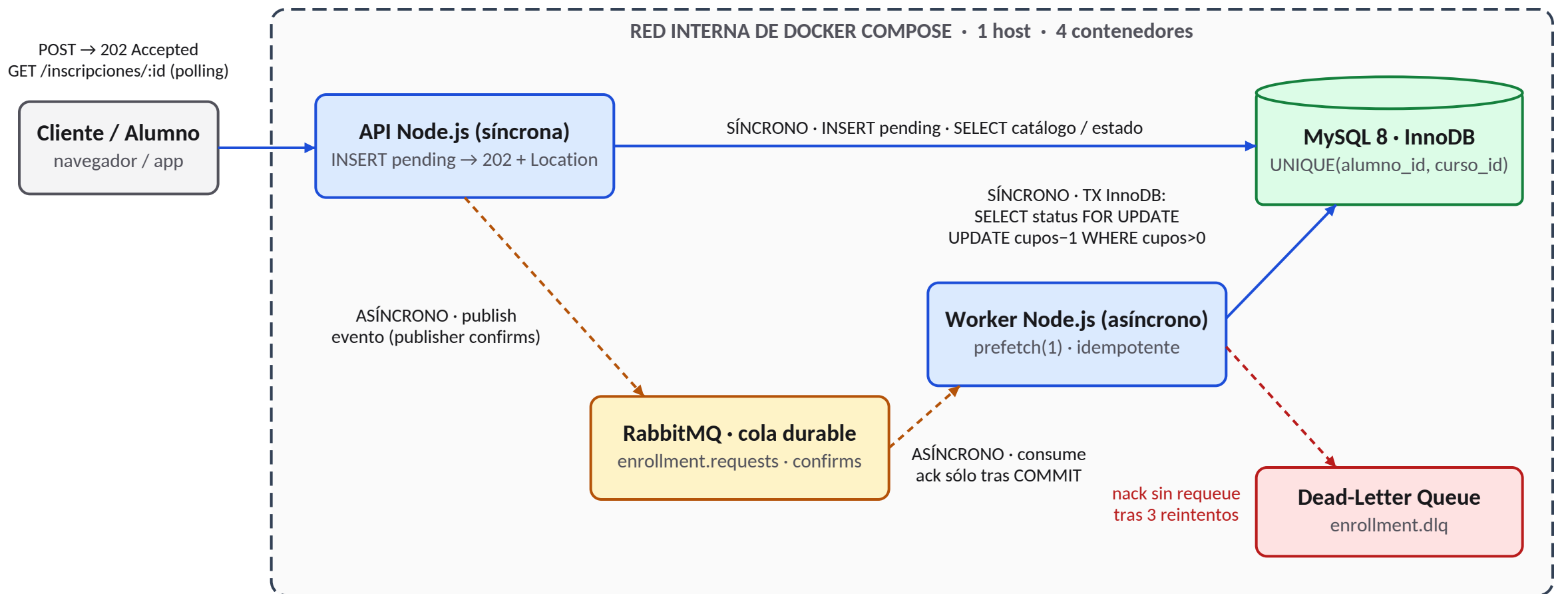
Si el worker muere antes del **ack**, RabbitMQ reentrega. Antes de tocar el cupo, el worker verifica bajo bloqueo de fila (**SELECT ... FOR UPDATE**) que la inscripción siga **pending**; si no, confirma y sale. **UNIQUE (alumno_id, curso_id)** bloquea el doble clic.

Tolerancia a fallos

Cola y mensajes durables · **publisher confirms** (si falla el publish → **failed** + 503) · **ack** solo después del COMMIT · reintentos acotados y **dead-letter queue** para mensajes venenosos · orden fijo de bloqueos (inscripciones → cursos) para evitar deadlocks.

Criterio de aceptación (prueba de carga, próximas entregas): 500 POST concurrentes contra un curso de 10 cupos → **exactamente 10 confirmados, 0 duplicados, 0 cupos negativos**.

Diagrama de despliegue



— Línea continua: síncrono - - - Línea punteada: asíncrono (fire-and-forget) [] Cóputo [] Mensajería [] Persistencia [] Dead-letter